COMPUTATIONAL METHOD

LAB FILE



MAHARAJA SURAJMAL INSTITUTE OF TECHNOLOGY

C4 , Janakpuri, New Delhi – 110058

Electronics and Communication Engineering Department

Name:- Ronit Chauhan

Class Serial No.:- 16

Enrollment No.:- 02315002825

Class:- ECE — 1

Submitted To:

Dr. Geetanjali

Dr. Deeba Naqvi

Practical : 1 (a)

Linear equation in 1 variable

Aim : Program to find value of x in Linear equation.

Code :

```
#include <stdio.h>
int main() {
   float a, b, x;
   printf("Equation: ax + b = 0\n");
   printf("Enter a and b: ");
   scanf("%f %f", &a, &b);
   if (a == 0) {
      printf("Not a linear equation (a=0).\n");
   } else {
      x = -b / a;
      printf("Solution: x = %.2f\n", x);
   }
   return 0;
}
```

Output:

```
linear equation : ax + b = 0
Enter a and b = 5 3
Solution: x = -0.600000
```

Practical : 1 (b)

Linear equation in 2 variable

Aim : Program to find value of x,y in Linear equation.

Code:

```
#include <stdio.h>
int main() {
   float a1, b1, c1, a2, b2, c2, x, y, eq1,eq2;
   printf("Equations: a1x + b1y = c1 & a2x + b2y = c2\n");
   printf("Enter a1, b1, c1: ");
   scanf("%f %f %f", &a1, &b1, &c1);
   printf("Enter a2, b2, c2: ");
   scanf("%f %f %f", &a2, &b2, &c2);
   //By applying elimination rule
   float denominator = (b1 * a2 - b2 * a1);

   if (denominator == 0) {
   printf("No unique solution (lines are parallel or coincident).\n");
}
   else {
   y = (c1 * a2 - c2 * a1) / denominator;
     if (a1 != 0) {
      x = (c1 - b1 * y) / a1;
     } else {
      x = (c2 - b2 * y) / a2;
     }

   printf("Solutions are x = %.2f, y = %.2f\n", x, y);
}
   return 0;
}
```

Practical : 1 (c)

Quadratic Equation

Aim : Program to find value of x in Quadratic equation.

Code:

```
#include <stdio.h>
#include <math.h>
int main() {
   float a, b, c, discriminant, x1, x2, realPart, imagPart;
   printf("Quadratic equation : ax^2 + bx + c = 0\n");
   printf("Enter value of a, b, c: ");
   scanf("%f %f %f", &a, &b, &c);
   discriminant = b * b - 4 * a * c;
   if (discriminant > 0) {
      x1 = (-b + sqrt(discriminant)) / (2 * a);
      x2 = (-b - sqrt(discriminant)) / (2 * a);
      printf("Roots are real and distinct: %.2f and %.2f\n", x1, x2);
   } else if (discriminant == 0) {
      x1 = -b / (2 * a);
      x2 = x1;
      printf("Roots are real and equal: %.2f and %.2f\n", x2, x1);
   } else {
      realPart = -b / (2 * a);
      imagPart = sqrt(-discriminant) / (2 * a);
      printf("Roots are complex: %.2f + %.2fi and %.2f - %.2fi\n", realPart, imagPart,
realPart, imagPart);
   }
   return 0;
}
```

Practical : 2

Bisection Method

Aim : Program for finding roots of equation x^3-x-5=0 using Bisection method.

Code:

```
#include <stdio.h>
#include <math.h>
double f(double x) {
  return x*x*x - x - 5;
}
int main() {
  double a = 2, b = 3, c, e = 0.000001;
  if (f(a) * f(b) > 0) {     // checking if intervals are valid or not
    printf("Invalid interval\n");
    return 0;
  }
  printf("Bisection Method for x^3 - x - 5 = 0\n");
  int iter = 1;
  while ((b - a) >= e){     // works only when value is greater than 0,0000001
    c = (a + b) / 2; // mid point
    printf("Iteration %d: x = %f\n", iter++, c);
    if (f(a) * f(c) < 0){     // if answer is positive sooo
      b = c;
    }
    else{         // if answer is negative soooo
      a = c;
    }
  };
  printf("Root is approximately: %f\n", c);
  return 0;
}
```

Practical : 3

Secant Method

Aim : Program for finding roots of of equation x^3-x-5=0 using Secant method.

Code:

```
#include <stdio.h>
#include <math.h>
double f(double x) {
   return x*x*x - x - 5;
}
int main() {
   double x0 , x1 , x2 ,e, iter;
   x0 = 1;
   x1 = 2;
   e = 0.000001;
   iter = 1;
   while (fabs(x1-x0) >= e)
   {  x2 = x1 - ((f(x1)*(x1-x0))/(f(x1) - f(x0)));
      x0 = x1;
      x1 = x2;
      iter++;
      printf("Iteration :%.6f | roots:x0 : %.6f x1 : %.6f \n",iter,x0,x1);
   }
   printf("roots:%.6f\n",x2);
   return 0;
}
```

Practical : 4

Newton Raphson Method

Aim: Program for finding roots of of equation x^3-x-5=0 using Newton Raphson method.

Code:

```
#include <stdio.h>
#include <math.h>
double f(double x) {
   return x*x*x - x - 5;
}
double df(double x){
   return 3*x*x - 1;
}
int main() {
   double x0 , x1 ,e, iter;
   x0 = 1;
   x1 = 2;
   e = 0.000001;
   iter = 1;
   while (fabs(x1-x0) >= e)
   {
      x0 = x1;
      x1 = x0 - f(x0)/df(x0);
      printf("Iteration :%.6f | roots:x0 : %.6f \n" , iter , x0);
      iter++;
   }
   printf("roots:%.6f\n",x1);
   return 0;
}
```

Practical : 5

Matrix Operations

Aim: Program to perform basic Matrix Operations.

- Addition
- Subtraction
- Multiplication
- Inverse
- Adjoint
- Determinant
- Eigenvalues
- Simultaneous Equations
- Orthogonal Check

Code:

```
#include <math.h>
#include <stdio.h>
#define Max 10
float determinant_calc(float a[Max][Max], int n) {
 if (n == 1) return a[0][0];
 if (n == 2) return a[0][0] * a[1][1] - a[0][1] * a[1][0];
 float det = 0, sub[Max][Max];
 int sign = 1;
 for (int x = 0; x < n; x++) {
  int subi = 0;
  for (int i = 1; i < n; i++) {
   int subj = 0;
   for (int j = 0; j < n; j++) {
    if (j == x) continue;
    sub[subi][subj++] = a[i][j];}
   subi++;}
  det += sign * a[0][x] * determinant_calc(sub, n - 1);
  sign = -sign;}
 return det;
}

void getCofactor(float a[Max][Max], float temp[Max][Max], int p, int q, int n) {
 int i = 0, j = 0;
 for (int row = 0; row < n; row++) {
  for (int col = 0; col < n; col++) {
   if (row == p || col == q) continue;
   temp[i][j++] = a[row][col];
   if (j == n - 1) { j = 0; i++; }}}}

void adjoint_calc(float a[Max][Max], float adj[Max][Max], int n) {
 if (n == 1) { adj[0][0] = 1; return; }
```

Practical : 6

Lagrange's Interpolation

Aim: Program to implement Langrange's Interpolation formula.

Code:

```
#include <stdio.h>
#include <stdlib.h>

double lagrange_interpolation(double* x_vals, double* y_vals, int n, double target_x) {
 double interpolated_y = 0.0;

 for (int i = 0; i < n; i++) {
  double basis_polynomial = y_vals[i];
  for (int j = 0; j < n; j++) {
   if (i != j) {
    basis_polynomial *= (target_x - x_vals[j]) / (x_vals[i] - x_vals[j]);
   }
  }
  interpolated_y += basis_polynomial;}
 return interpolated_y;
}

int main() {
 int n;
 double target_x;

 printf("Enter the number of data points (n): ");
 if (scanf("%d", &n) != 1 || n <= 0) {
  printf("Error: Number of points must be a positive integer.\n");
  return 1;}

 double* x_vals = (double*)malloc(n * sizeof(double));
 double* y_vals = (double*)malloc(n * sizeof(double));

 if (x_vals == NULL || y_vals == NULL) {
  printf("Error: Memory allocation failed!\n");
  return 1;}

 printf("\nEnter the data points (x and corresponding y):\n");
```